AI 401 @ Workplace

2026

Henrique Sanchez
2026-10-14

Recap...

Where we've been:

- **AI 101 - AI 301:** Chatbots

Where you are now:

- You've been doing reliable work out of chatbots
- You encounter fewer issues day to day out of small tasks
- You're feeling the frustration of trying to do larger jobs and tasks
- You're tired of copy/pasting prompts and results from one instance/session to another

The Past, Present, and (Near) Future...

Most are still here!

We are here!

When	Stage	What It Means	What You Can Do
Early 2023	Basic Chatbots	AI answers questions from memory	Ask questions, get text answers (often wrong)
Mid 2023	Document Attachments	AI can read your files	"Summarize this PDF" or "Analyze this spreadsheet"
Late 2023	Custom Bots + Knowledge Bases	AI trained on your specific docs	Build a "company expert" that knows internal policies
2024	Tool-Using Agents	AI can take actions, not just talk	Autonomously send emails, query databases, call APIs
Early 2025	Looping Agents	AI works autonomously in cycles	"Build this feature" → AI codes, tests, fixes, repeats until it's done, navigating obstacles
Late 2025–2026	Orchestrators & Swarms	Multiple specialized AIs coordinate	Teams of agents: one plans, one researches, one executes

AI as a **tool**

AI as a **colleague**

What we'll cover today

1. **Upgrading to Agents** (What Actually Changes)
2. **Harness Engineering** (What Really Unlocks Agentic Workflows)
3. **Determinism in Agentic Workflows** (Reliable Outputs From Inputs)
4. **Add-ons** (Improving Usefulness of the Agentic Harness)
5. **Introducing Loops** (Small Steps for Long-Horizon Tasks)
6. **The Translation Barrier** (Transferring Human Knowledge & Wisdom)

Don't worry about memorizing anything, all materials and examples will be available in the description below. Focus on following the logic and understanding the *hows* and the *whys*

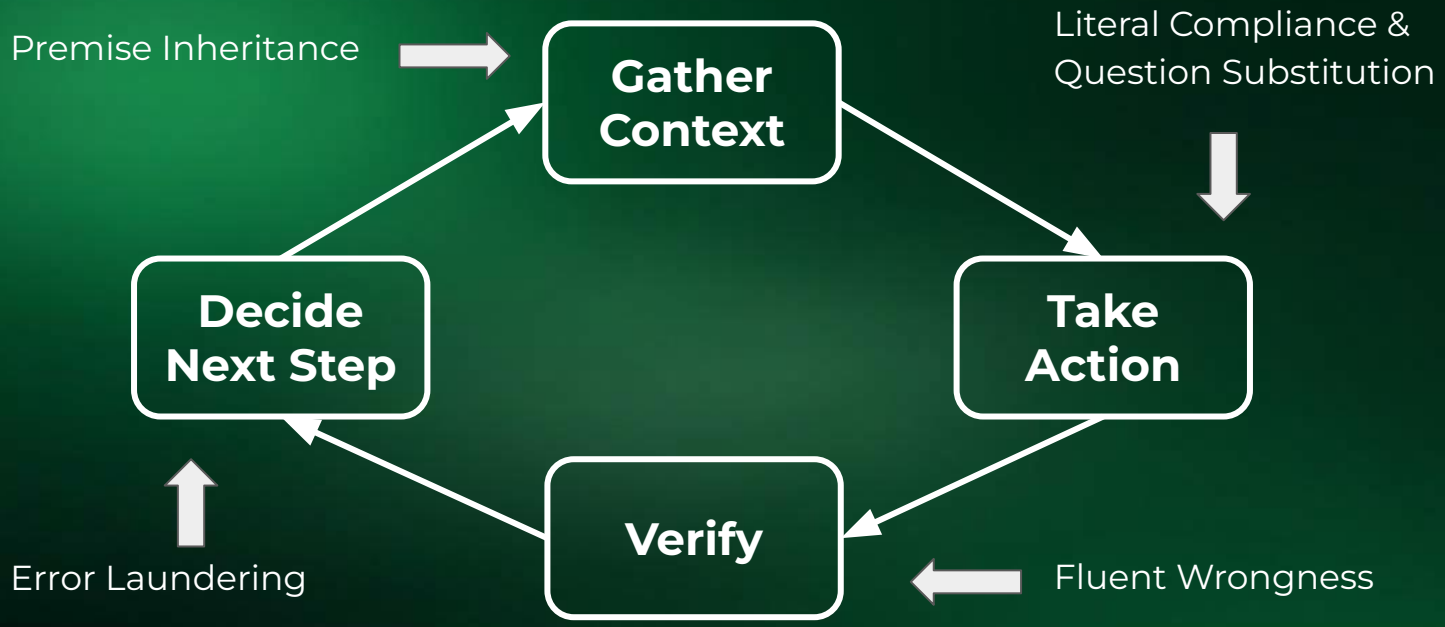
1. Upgrading to Agents: What actually changes

Remember from AI 301: "An agent is the same LLM you have been using all along, wrapped in a harness that gives it tools and lets it run in a loop until the job is done."

- Chatbot = you are between every turn (you read, you paste, you press Enter)
- Agent = you set the goal and the guardrails, then you are *on* the loop, not *in* it.

In 301 you **were** the check, and now in 401, you **build** the checks.

1. Upgrading to Agents: The agent loop



Every box is a place we can put a guardrail. That's what the rest of today is about.

1. Upgrading to Agents: The mental model

Wrong mental model: a better agent needs a better model.

Right mental model: Agent = Model + Harness. The model is the talent, and the harness is everything else: instructions, tools, permissions, memory, checks.

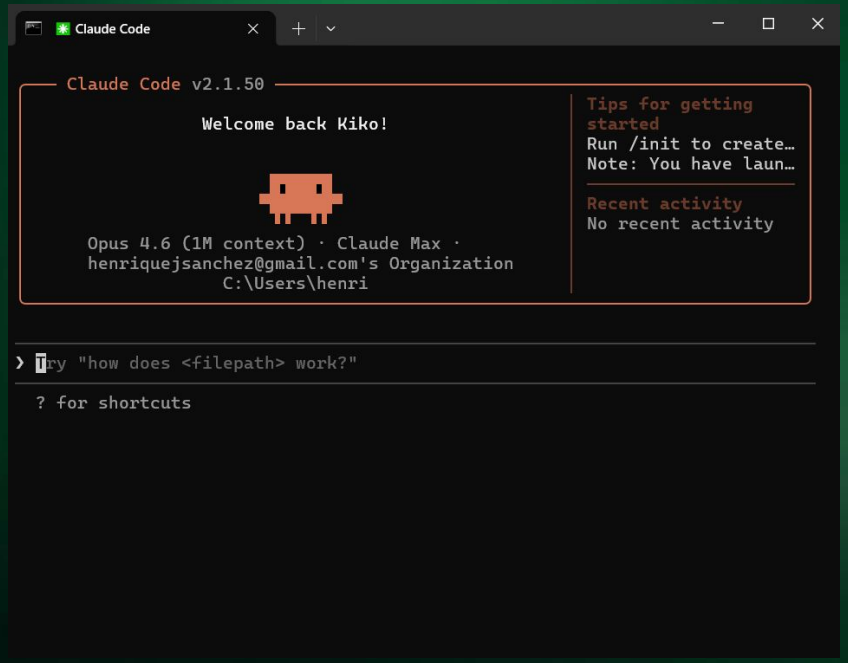
A new hire needs...	An agent needs...	Where it lives
Onboarding pack	Instruction file (AGENTS.md / CLAUDE.md)	Section 2
A desk and an access badge	Workspace folder + permissions	Section 2
Checklists and QA	Lints, tests, hooks, scripts	Section 3
Training on procedures	Skills	Section 4
Logins to company systems	MCP connectors	Section 4
A way to work through a long project	Loops with notes in files	Section 5
Your experience, written down	A translated knowledge library	Section 6

1. Upgrading to Agents: The different cockpits

Interface	What it is	What it feels like	Examples (as of Oct 2026)
GUI (graphical)	Desktop or web app with buttons, file previews, approvals	A chat app that can open your folders and act	Claude desktop app, ChatGPT / Codex app, Cursor and other AI editors
TUI (text user interface)	A full-screen app that runs inside the terminal	A chat window, but in a terminal, with menus and keyboard shortcuts	Claude Code, Codex CLI, Gemini CLI, OpenCode, Pi.dev, GitHub Copilot CLI
CLI (command line)	One command in, result out; no conversation	A tool you call from scripts and schedules	`claude -p "…"`, `codex exec "…"`, any agent in "headless" mode

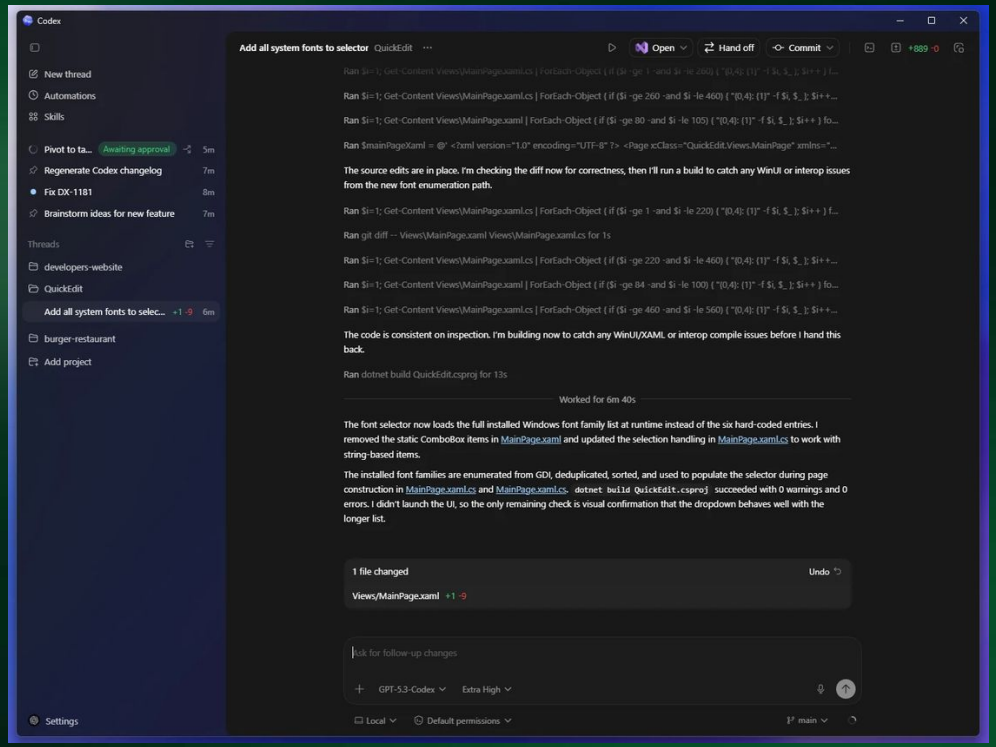
Key point: **all three drive the same model through a harness**. The interface changes how much you see and how easily you can automate, not how smart it is.

1. Upgrading to Agents: The different cockpits



Example of TUI/CLI

Example of GUI



1. Upgrading to Agents: Which cockpit for which jobs?

Your options:

- **GUI** when: you're starting out, you work with documents, you want to see and approve each action. Lowest barrier for non-developers.

- **TUI** when: you want speed, keyboard control and full access to hooks, skills and settings. The terminal is just a text box with superpowers, while you type in plain language.

- **CLI / headless** when: the task should run the same way every time, from a script or a schedule (Sections 3 and 5).

Practical rule: start in the GUI, graduate to the TUI when you hit its limits, reach for the CLI when you want repeatability.

Reminder from 101: check which tools IT has vetted. The interface choice is often made for you, while the concepts transfer easily.

1. Upgrading to Agents: Folders and directories

With a chatbot, the conversation was the workspace. With an agent, a **folder is the workspace** and the conversation is disposable.

Example:

```
monthly-supplier-report/

  AGENTS.md            ← who the agent is, the rules, where things are

  inputs/              ← raw files dropped here (read-only for the agent)

  outputs/             ← the only place it may write

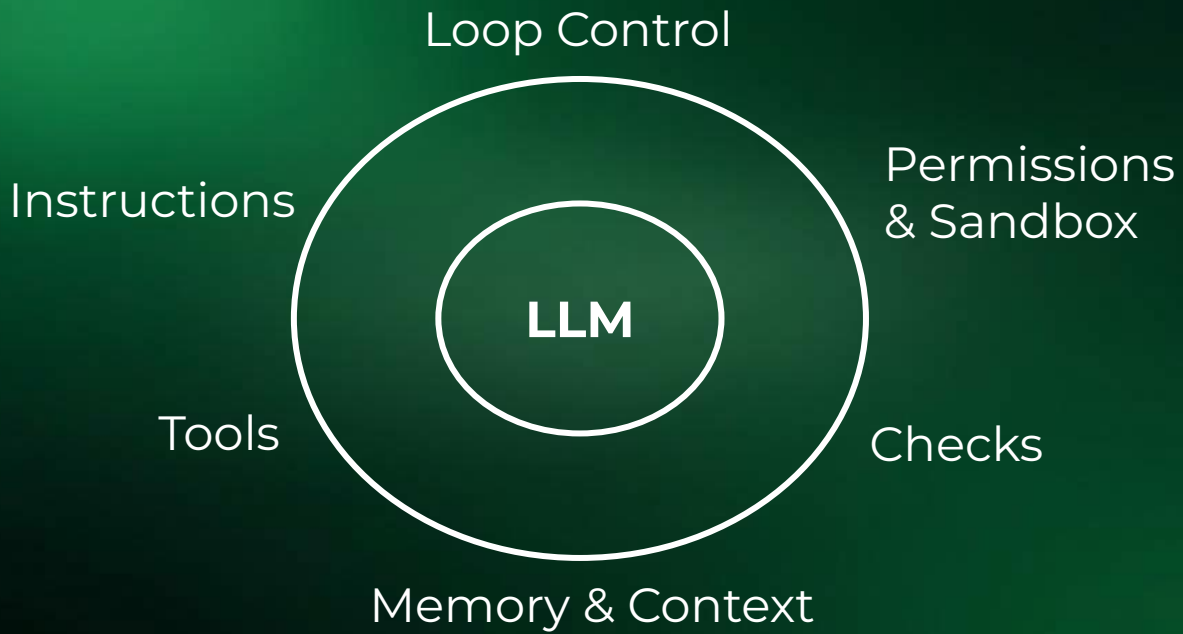
  scripts/             ← deterministic helpers (Section 3)

  checks/              ← your 301 eval, as a list or a script

  notes/progress.md    ← its working memory across sessions
```

2. Harness Engineering: What is a harness?

Definition: the harness is everything around the model that turns it into an agent. If it isn't the model, it's the harness.



Prompt engineering was about one message. Context engineering was about one window. **Harness engineering** is about the whole workplace

2. Harness Engineering: Why the harness matters

The harness around an LLM matters more than the next model release:

- Whenever the agent makes a mistake, you can build something so it never makes that mistake again (e.g.: better instruction files, and real tools that tell it what's wrong)
- You can make repositories legible to the agent and add missing tools, guardrails, and docs whenever it struggles
- Real world example: LangChain moved its coding agent from outside the top 30 to the top 5 on Terminal Bench 2.0 (52.8% → 66.5%) by changing only the harness and using the same model.

2. Harness Engineering: The instruction file (AGENTS.md)

AI 301's "system prompt" now lives **in the folder** as a plain text file the agent reads at the start of every session. **`AGENTS.md`** is the cross-tool convention; Claude Code reads **`CLAUDE.md`** (many teams just point one at the other).

Example: Monthly supplier report

```
Purpose: turn the files in inputs/ into outputs/report.md for the ops manager.
Never edit anything in inputs/. Write only to outputs/.
Numbers: every figure must cite file + page. If unsure, write UNVERIFIED.
Use scripts/extract_tables.py for tables; do not read tables by eye.
Before finishing, run scripts/check_report.py and fix every failure.
Units: m³ and €, Swedish date format (YYYY-MM-DD).
Tone: plain, no buzzwords, max 1 page summary.
Known traps: supplier B reports in thousands; Q-codes are not quantities.
Progress notes go in notes/progress.md.
```

Three rules: keep it short (a **map**, not an encyclopedia; link out to detail files), write each line because of a ***real*** mistake you saw, and review it monthly and delete what no longer applies.

2. Harness Engineering: Permissions, sandboxes and undo

Dial	Safe default	Loosen when…
What it can read	Only the task folder	It needs a reference library (add that folder read-only)
What it can do without asking	Read, search, write to `outputs/`	You've watched it do the same action correctly many times
What it can never do	Delete, send messages, touch originals, spend money	Never on the agent's say-so; only by your explicit change

Undo is a harness feature. Work on copies, keep version history (git, or the versioning of your document system), so any run can be rolled back.

Privacy and IT rules still apply. A sandbox is also where you keep sensitive data *out*.

2. Harness Engineering: Automated context engineering

AI 301 taught manual context moves: rewind, parallelize, handoff, externalize. Now we can automate it:

- **Handoff** → automatic compaction / summarizing when the window fills
- **Externalize** → the agent writes `notes/progress.md` and reads it next session
- **Rewind** → checkpoints and version history
- **Parallelize** → several attempts or reviewers, via subagents

New idea: **progressive disclosure**. Don't load everything up front. Instead, the instruction file says where things are, and the agent opens a file only when the task needs it. This keeps the agent in the "smart zone" of its context window.

The dumbzone curve didn't go away. The harness just manages it for you, if you design it to.

2. Harness Engineering: The harness improvement loop

The mistake was...	Fix it with...	Section
It didn't know a rule	One line in the instruction file	2
It knew the rule but ignored it sometimes	A script, lint or hook that checks it every time	3
It did a multi-step procedure badly	A skill with the procedure written out	4
It couldn't reach the information	A connector, or a file in the folder	4
It lost track on a long task	Smaller tasks in a loop with notes in files	5
It lacked your judgment	Translation work: write the judgment down	6

Rule: Never fix the same mistake twice by hand

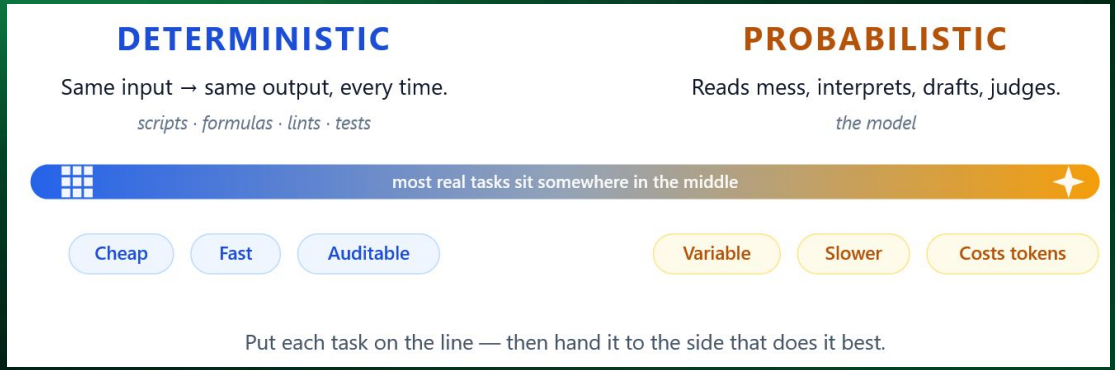
2. Harness Engineering: Your harness starter kit

- [ ] One folder per recurring task, with `inputs/` (read-only) and `outputs/`
- [ ] An instruction file under ~40 lines, written from real mistakes
- [ ] Version history turned on, so every run can be undone
- [ ] Permissions set to "ask" for anything that deletes, sends or spends
- [ ] A `notes/progress.md` the agent updates at the end of each session
- [ ] A mistake log: date, what went wrong, which harness fix you made
- [ ] Your 301 eval saved as `checks/` (Section 3 will make it executable)

3. Determinism in Agentic Workflows

Remember AI 101: LLMs are a probabilistic engine: same input, similar but not identical output. That was a warning in 101. In 401 it becomes a design rule.

Think of it as a spectrum:



The question to ask of every step in a workflow: "Does this step have one right answer?" If yes, it belongs on the left.

Examples of steps that should never be left to chance: totals and conversions, date formats, file naming, "did every section get filled", "does every number have a source".

3. Determinism in Agentic Workflows

Let the AI write the script, then run the script

The most underrated move in agentic work is that instead of asking the agent to do a repetitive step every month, ask it to write a script that does the step, check the script once, and reuse it forever.

Ask the agent to do it each time	Ask the agent to write a script once
Different result each run, small drift	Same result every run
You re-check every output	You check the script once (plant an error to test it)
Costs tokens and minutes every time	Runs in seconds, near-free
Hard to audit	The script *is* the audit trail

Workplace example: "Extract the flow readings table from these 30 lab PDFs into a spreadsheet." Month 1: the agent writes `extract_readings.py` . Month 2 onward: the agent runs the script and spends its effort on the anomalies.

3. Determinism in Agentic Workflows: The toolbox

Tool	What it is	When it runs	Workplace example
Script (bash, Python)	A small program that does one job the same way every time	When you or the agent call it	Rename and file 200 scanned invoices by supplier and date
Lint / check	A script whose only job is to say pass or fail, with a reason	Before the work is accepted	"Every figure in the report cites a source file"
Hook	A script the *harness* runs automatically at a set moment in the agent loop	Every time that moment happens; the agent can't skip it	Block any edit to `inputs/`; run the checks whenever it tries to finish
Scheduled job (cron)	A command that runs on a timetable	Every Monday 07:00, first of the month…	Pull last week's exports into `inputs/` before you arrive

Scripts do work. Checks judge work. Hooks enforce checks. Schedules start work.

3. Determinism in Agentic Workflows: Lints & Checks

301 eval line (you check)	401 check (a script checks)
The three vendors are the three I named	Script compares vendor names in output against `brief.md`
Every number traces to the source table	Script finds every number and confirms it appears in `inputs/`
The risk caveat survived the shortening	Script fails if the section "Risks" is missing or empty
It answers "which fits our constraints"	**Stays human** (or a fresh-chat critic): this is judgment
I'd be fine if the client forwarded this	**Stays human**: this is taste

3. Determinism in Agentic Workflows: Hooks

An instruction can be ignored. A hook can't.

Three moments where hooks fire (remember the agent loop):

1. Before an action — can block it. E.g. refuse any write outside `outputs/`, refuse any delete, refuse sending email.
2. After an action — runs a check and feeds the errors back to the agent. E.g. after every edit to the report, run the number-tracing check.
3. When the agent tries to finish — can say "not yet". E.g. it can't declare done until `check_report.py` passes.

3. Determinism in Agentic Workflows: Scheduled Jobs

"Cron" is the classic Unix scheduler (Windows Task Scheduler is the same idea; many agent apps now offer "scheduled tasks"). It answers when, not what.

Pattern for AI 401: schedule the deterministic part, and review the AI part. Example:

- 06:45 — script pulls last week's exports into `inputs/` and runs the input checks
- 07:00 — agent drafts the weekly summary into `outputs/`, checks run via hooks
- 09:00 — you read a draft that already passed every machine check, and apply judgment

Scheduled runs get the tightest permissions (no sending, no deleting), because nobody is watching.

Letting it run without you reading every output is AI 501 aka going “hands-free".

3. Determinism in Agentic Workflows: The Sandwich



Rule of thumb: *keep the probabilistic layer as thin as it can be while still doing the work only a model can do.*

Determinism doesn't make the model smarter. But it does make its mistakes cheap, visible and early.

4. Add-ons

Add-on	Analogy	What it gives the agent	One-line test
Skill	The procedure binder on the shelf	*Know-how*: how we do a specific task here	"It knows the tool but not our way of doing it"
MCP connector	A login to a company system	*Access*: read or act in another system	"It can't see or reach the thing it needs"
Plugin	The onboarding box for a whole team	*Packaging*: skills + hooks + connectors + commands, installed together	"Five people need the same setup"

The instruction file says how to behave in this folder. Add-ons travel with you to every folder.

4. **Add-ons:** Skills

What it is: a folder with a `SKILL.md` file (a name, a one-line description of when to use it, then the procedure), plus optional scripts, templates and examples.

Why it doesn't flood the context: only the name and description are loaded up front. The full procedure is opened only when the task matches.

Portable: Anthropic introduced skills in October 2025 and released them as an open standard (agentskills.io) in December 2025. They now work across Claude Code, OpenAI Codex, Gemini CLI, GitHub Copilot, Cursor and dozens of other tools.

4. Add-ons: Skills

```
name: incident-report

description: Use when writing a plant incident report from
operator logs and alarm exports.

1. Run scripts/parse_alarms.py on the alarm export. Never
   summarize alarms by eye.
2. Build the timeline only from timestamps in the logs; mark
   gaps as GAP.
3. Fill templates/incident.md. Root cause stays "Under
   investigation" unless the logs state it.
4. Run scripts/check_incident.py; fix every failure before
   finishing.
```

Note how the skill contains determinism (Section 3): the good ones are 30% instructions, 70% "run this script".

4. Add-ons: MCP (Model Context Protocol)

What it is: an open standard plug that lets any agent connect to any system that offers an MCP server: document stores, ticketing, CRM, databases, calendars. Often called "USB-C for AI".

What it means at work: instead of exporting a spreadsheet and uploading it, the agent queries the system directly, with your permissions.

Two cautions:

- Every connected server adds its tool descriptions to the context. Ten connectors you don't use today push you toward the AI’s context dumbzone. Connect only what the task needs.

- A connector that can write or send is an action with consequences. Start read-only.

4. Add-ons: Plugins

What it is: one installable bundle that can hold skills, hooks, slash commands, connector settings and agent definitions. Claude Code and Codex call them plugins; Gemini CLI calls them extensions.

Why it matters for teams: it turns "the way our team works with agents" into something you install in one step and update in one place. Version it like any other shared tool.

Team marketplace idea: a shared repository of your team's plugins (e.g. `ops-reporting`, `procurement-review`), owned by someone, reviewed like code.

Example contents of an ops-reporting plugin:

- Skills: `weekly-summary`, `incident-report`
- Hooks: block edits to raw data; run checks before finishing
- Connector config: read-only access to the document store
- Command: `/weekly` to start the weekly run

4. Add-ons: Choosing and staying safe

Missing a rule? → instruction file.
Missing a procedure? → skill.
Missing access? → connector.
Missing consistency across people? → plugin.

The lethal trifecta (Simon Willison, 2025): an agent becomes exploitable when it has all three of:

1. access to private data,
2. exposure to untrusted content (emails, web pages, supplier PDFs),
3. a way to communicate externally (send, post, upload).

Rule: Never give one agent all three without a human approval in between. Models still follow instructions hidden in content

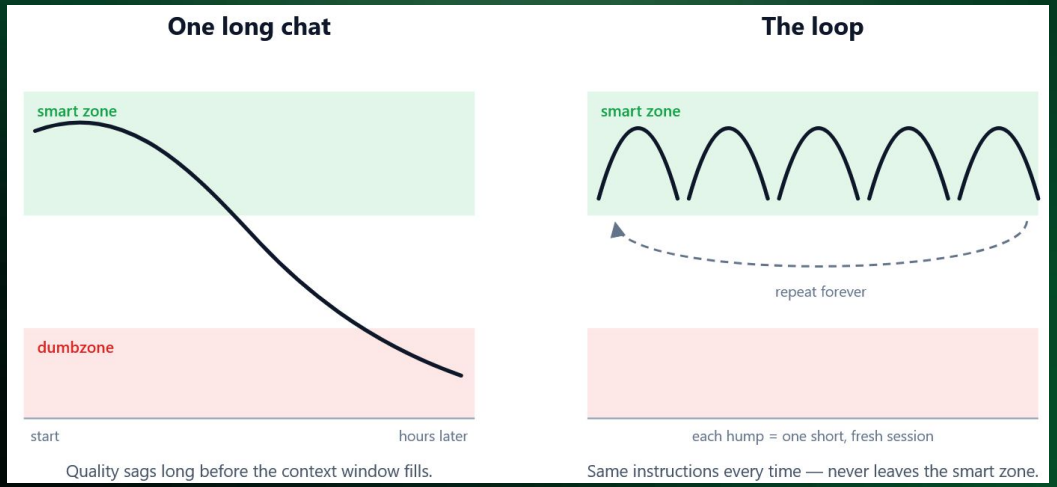
Practical rules: third-party skills and plugins are code, so read them before installing; prefer add-ons your IT department has vetted; start every connector read-only.

5. Introducing Loops

The problem: big jobs don't fit in one context window. In AI 301 we showed quality drops long before the window is full.

The manual fix was that restart beats repair, and context must be computed, not accumulated, by keeping memory in a doc you own.

A loop is that same fix, but automated. You do this by implementing many short fresh sessions, each doing one small step, each reading and updating the same files.



5. Introducing Loops: Ralph Wiggum

```
while :; do cat PROMPT.md | claude -p ; done
```

"*Forever: give the agent the same instructions, let it do one piece of work, start again with a clean head*".

Origin: Geoffrey Huntley published it in July 2025 and named it after the Ralph Wiggum character from The Simpsons (Ralph being persistent, but not clever). By 2026 it had become a standard pattern in workflows.

Why does it work?

- Fresh context every iteration (remember the “smart zone”)
- State in files, not in the chat (remember “externalizing”)
- Backpressure from checks rejects bad work

5. Introducing Loops: Anatomy of a (Safe) Loop

1. A spec: what "done" looks like, written down
2. A task list file: every item to process, with a checkbox. The loop's to-do list lives on disk.
3. One task per iteration: small steps keep each run in the smart zone and make failures easy to locate.
4. Backpressure: the checks and hooks from Section 3 run every iteration; failing work is not ticked off.
5. Progress notes: each iteration appends what it did and what it found odd to `notes/progress.md`.
6. Stop conditions: a completion signal (all boxes ticked), an iteration cap, and "stop and write a question" when stuck.

5. Introducing Loops: Example of a workplace Loop

Example: 150 supplier quality certificates → a compliance register.

- `tasks.md`: 150 lines, one per certificate, each with [ ]
- `PROMPT.md` (same every iteration):

```
Read AGENTS.md and notes/progress.md.
Take the FIRST unchecked item in tasks.md. Only that one.
Run scripts/extract_cert.py on it. Add one row to outputs/register.csv.
Run scripts/check_register.py. If it fails, fix your row; do not tick the
item.
If the certificate is expired, unreadable or odd, add it to
outputs/exceptions.md.
Tick the item, append one line to notes/progress.md, then stop.
If every item is ticked, write ALL DONE.
```

- Guardrails: inputs read-only, no sending, max 200 iterations, run during the day the first time and watch the first ten.

5. Introducing Loops: When to Loop

Good fit for a loop	Bad fit for a loop
Many similar items	One big judgment call
"Done" is machine-checkable	"Done" is a matter of taste
Each step is reversible	Steps send, delete, pay or publish
Inputs are trusted documents	Inputs are untrusted (emails, web) plus private data (lethal trifecta)

Cost note: every iteration spends tokens. Use a cap, start with a small batch, and look at the bill after the first run.

6. The Translation Barrier

	BAU (business as usual)	AI-assisted work	AI-optimized work
Who does the work	People	People, with a chatbot or agent helping	Agents in a harness; people set goals and judge
What it requires	Skills and time	**Only that AI tools exist** (a license)	**Translation**: knowledge turned into files, plus a shared library
Where know-how lives	In heads, emails, habits	Still in heads; pasted into prompts each time	In instruction files, skills, scripts, checks
Repeatability	Depends on who's in	Depends on who prompts	Same result whoever starts the run
Courses that cover it	—	AI 101 – 301	AI 401 onward

AI-assisted work arrives when you buy the tools. AI-optimized work arrives only when you do the translation

6. The Translation Barrier: Tacit Knowledge

Philosopher Michael Polanyi (1966): "*We can know more than we can tell*".

Experts routinely follow rules they've never written down, and often can't fully state.

Typical tacit knowledge in any workplace:

- "Supplier B always reports in thousands."
- "Never schedule that maintenance on a Monday; the weekend backlog is still running."
- "The board wants the bad news in the first paragraph."

Remember AI 301's literal-minded colleague analogy: "Did you forget to mention a constraint? The agent will not infer the constraint."

Translation is the act of mentioning every constraint, once, in a place the agent will read.

6. **The Translation Barrier**: How to translate

What you produce (your team's library):

Artifact	**Holds**
Instruction files	Rules and context for a workspace
Scripts and checks	Everything with a right answer; your evals
Skills	Procedures, step by step
Plugins	The bundle the whole team installs
Structured reference docs	Facts, limits, glossaries, in the "structured" format
Decision log	What was decided and **why**

6. **The Translation Barrier**: How to translate

How you extract it (four techniques):

1. **Be interviewed by the agent**. "Ask me one question at a time about how I do this task until you could do it without me. Then write the skill." (AI 201's clarification technique, pointed at yourself.)

2. **Narrate while you work**. Record yourself doing the task once, talking through every decision; have the agent turn the transcript into a draft procedure.

3. **Mine your mistake log**. Every harness fix from Section 2 is a piece of translated knowledge.

4. **Restructure old documents**. Turn long prose paragraphs into constraints and procedures (AI 301 showed this; the agent can do the first pass).

Rule: every library item gets an **owner** and a **review date**, or it rots.

6. **The Translation Barrier**: Your translation sprint

[ ] **Week 1: Workspace**. Pick one recurring task. Give it a folder, an instruction file and version history (Section 2).

[ ] **Week 2: Know-how**. Get interviewed by the agent; turn the answers into one skill (Sections 4 and 6).

[ ] **Week 3: Determinism**. Ask the agent to write the scripts and checks; add one hook that makes the checks unskippable (Section 3).

[ ] **Week 4: Run and measure**. Run it for real, log every mistake, fix the harness, and share it with one colleague as a plugin.

How to judge success: **How many times did you have to step in per run, week 1 vs week 4?** That number falling is what AI-optimized looks like.

Your harness, written down

- AI 201: Your LLM is only as good as your ability to direct it.
- AI 301: Your output is only as trustworthy as your ability to check it.
- AI 401: Your agent is only as reliable as the harness you build around it.

In AI 301 your judgment moved from your head to a note. In AI 401 it moved from a note into the workspace: instruction files, scripts, hooks, skills. That's what lets something work while you're not between the turns.

Building the foundations

With time, models and harnesses will improve, hallucinations will reduce, hand-holding will be less needed, guardrails and checks less necessary, and we will trust the human + AI agent system more.

But in the meantime, we continue to build the foundation of how to work together with AI in an effective and productive way, by understanding the fundamentals of what they are and what they enable.

